

Current Project Status

Backend

- ✓ Vehicle Simulator
 - ✓ SocketCAN (vcan0)
 - ✓ MQTT Broker
 - ✓ JSON Telemetry
 - ✓ SQLite Logger
-

Frontend

- ✓ Streamlit Dashboard
- ✓ Live Metrics
- ✓ GPS Coordinates
- ✓ Historical Charts
- ✓ Auto Refresh
- ✓ Timestamped Charts

The architecture now looks like a mini production telematics stack:

None

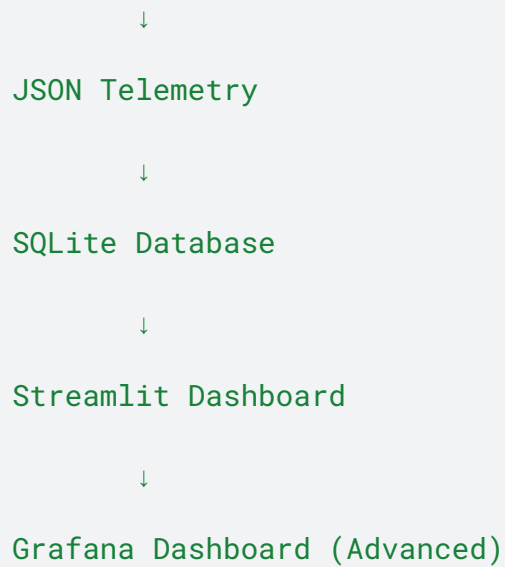
Vehicle Simulator (C++)

↓

SocketCAN (vcan0)

↓

MQTT Broker (Mosquitto)



Step 1: Install Streamlit

Verify:

```
None  
streamlit --version
```

If not installed:

```
None  
pip3 install streamlit pandas plotly
```

Verify:

```
None  
streamlit --version
```

Streamlit 1.58.0

1.1. Dashboard does not refresh automatically

Install:

```
None  
pip3 install streamlit-autorefresh
```

Step 2: Create Dashboard File

```
None  
nano tcu_dashboard.py
```

Paste:

TCU_dashboard.py

```
import streamlit as st  
import sqlite3  
import pandas as pd  
  
import plotly.express as px  
  
from streamlit_autorefresh import st_autorefresh  
  
# -----  
# Page Config  
# -----  
  
st_autorefresh(  
    interval=1000,  
    key="dashboard_refresh"  
)  
  
st.set_page_config(  
    page_title="TCU Dashboard",
```

```

        layout="wide"
    )

    st.title("Linux-Based TCU Dashboard")

    # -----
    # Database Connection
    # -----

    conn = sqlite3.connect(
        "vehicle_telematics_v2.db"
    )

    # -----
    # Latest Data
    # -----

    latest = pd.read_sql_query(
        """
        SELECT *
        FROM telematics
        ORDER BY id DESC
        LIMIT 1
        """,
        conn
    )

    if not latest.empty:

        row = latest.iloc[0]

        col1, col2, col3, col4 = st.columns(4)

        col1.metric(
            "Speed",
            f'{row["speed"]} km/h'
        )

        col2.metric(
            "RPM",
            int(row['rpm'])
        )

        col3.metric(

```

```

        "Fuel",
        f"{row['fuel']} %"
    )

col4.metric(
    "Engine Temp",
    f"{row['engine_temp']} C"
)

st.markdown("---")

gps1, gps2 = st.columns(2)

gps1.metric(
    "Latitude",
    f"{row['latitude']:.4f}" # row["latitude"] provides values more precision like
80.27230000000006 so lets round off to 4 decimal point using f"{row['latitude']:.4f}"
)

gps2.metric(
    "Longitude",
    f"{row['longitude']:.4f}"
)

# -----
# Historical Data
# -----

df = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC
    LIMIT 100
    """,
    conn
)

df = df.sort_values("id")
df["timestamp"] = pd.to_datetime(df["timestamp"])

st.markdown("---")

#-----SPEED GRAPH-----

```

```
st.subheader("Speed History")
```

```
fig_speed = px.line(  
    df,  
    x="timestamp",  
    y="speed",  
    title="Speed vs Time"  
)
```

```
st.plotly_chart(  
    fig_speed,  
    use_container_width=True  
)
```

```
#-----RPM GRAPH-----
```

```
st.subheader("RPM History")
```

```
fig_rpm = px.line(  
    df,  
    x="timestamp",  
    y="rpm",  
    title="RPM vs Time"  
)
```

```
st.plotly_chart(  
    fig_rpm,  
    use_container_width=True  
)
```

```
#-----FUEL GRAPH-----
```

```
st.subheader("Fuel History")
```

```
fig_fuel = px.line(  
    df,  
    x="timestamp",  
    y="fuel",  
    title="Fuel vs Time"  
)
```

```
st.plotly_chart(  
    fig_fuel,  
    use_container_width=True  
)
```

```
#-----ENGINE TEMP HISTORY-----
```

```
st.subheader("Engine Temperature History")
```

```
fig_temp = px.line(  
    df,  
    x="timestamp",  
    y="engine_temp",  
    title="Engine Temperature vs Time"  
)
```

```
st.plotly_chart(  
    fig_temp,  
    use_container_width=True  
)  
conn.close()
```

Step 3: Run Dashboard

Terminal 1:

None

```
python3 vehicle_logger_json.py
```

Terminal 2:

None

```
./tcu_1publisher_mqttCAN
```

Terminal 3:

None

```
streamlit run TCU_dashboard.py
```

You would see:

None

Local URL:

<http://localhost:8501>

Open:

None

<http://localhost:8501>

Linux-Based TCU Dashboard

Speed

119 km/h

RPM

4192

Fuel

18 %

Engine Temp

96 C

Latitude

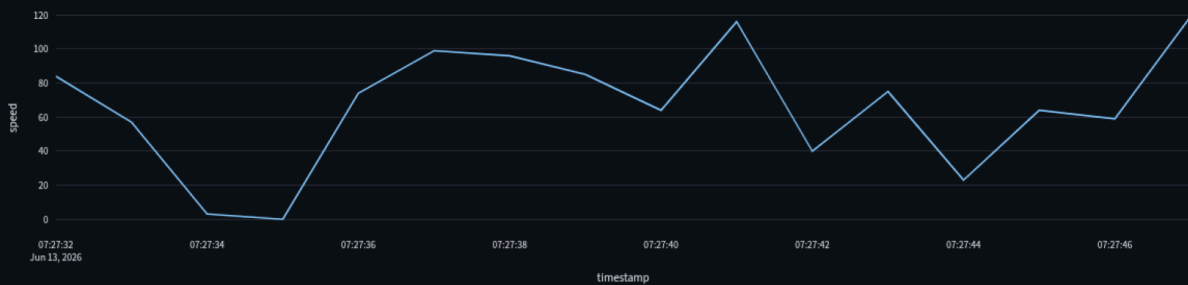
13.084299999999997

Longitude

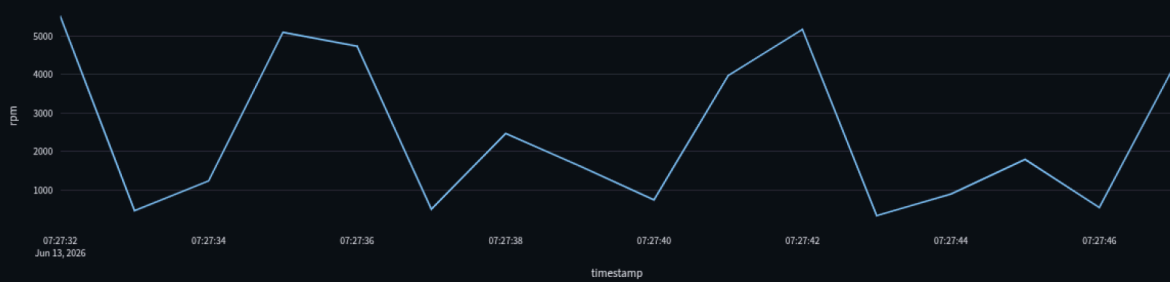
80.272300000000006

Speed History

Speed vs Time



RPM vs Time





Adding Instrument Cluster to dashboard - Version 1

- ✓ Auto refresh every second
- ✓ Instrument Cluster Gauges (Speed, RPM, Fuel, Temperature)
- ✓ Vehicle Health Indicator
- ✓ GPS Display
- ✓ Historical Plotly Charts with timestamps
- ✓ Existing metrics retained

TCU_dashboard_v1.py

```
import streamlit as st
import sqlite3
import pandas as pd
```

```
import plotly.express as px
```

```

import plotly.graph_objects as go

from streamlit_autorefresh import st_autorefresh

# -----
# Page Config
# -----

st_autorefresh(
    interval=1000,
    key="dashboard_refresh"
)

st.set_page_config(
    page_title="TCU Dashboard",
    layout="wide"
)

st.title("Linux-Based TCU Dashboard")

# -----
# Database Connection
# -----

conn = sqlite3.connect(
    "vehicle_telematics_v2.db"
)

# -----
# Latest Data
# -----

latest = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC

```

```

LIMIT 1
"""
conn
)

if not latest.empty:

    row = latest.iloc[0]

    # =====
    # INSTRUMENT CLUSTER
    # =====

    st.header("Instrument Cluster")

    speed_col, rpm_col = st.columns(2)

    # -----
    # Speed Gauge
    # -----

    with speed_col:

        fig_speed_gauge = go.Figure(
            go.Indicator(
                mode="gauge+number",
                value=row["speed"],
                title={"text": "Speed (km/h)"},
                gauge={
                    "axis": {"range": [0, 120]}
                }
            )
        )

    st.plotly_chart(
        fig_speed_gauge,
        use_container_width=True

```

```

    )

# -----
# RPM Gauge
# -----

with rpm_col:

    fig_rpm_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["rpm"],
            title={"text": "RPM"},
            gauge={
                "axis": {"range": [0, 6000]}
            }
        )
    )

    st.plotly_chart(
        fig_rpm_gauge,
        use_container_width=True
    )

# -----
# Fuel + Temperature Gauges
# -----

fuel_col, temp_col = st.columns(2)

with fuel_col:

    fig_fuel_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["fuel"],
            title={"text": "Fuel (%)"},

```

```

        gauge={
            "axis": {"range": [0, 100]}
        }
    )
)

st.plotly_chart(
    fig_fuel_gauge,
    use_container_width=True
)

with temp_col:

    fig_temp_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["engine_temp"],
            title={"text": "Engine Temp (C)"},
            gauge={
                "axis": {"range": [80, 120]}
            }
        )
    )

    st.plotly_chart(
        fig_temp_gauge,
        use_container_width=True
    )

# =====
# VEHICLE HEALTH
# =====

st.markdown("---")
st.subheader("Vehicle Health")

if row["engine_temp"] > 100:

```

```

        st.error(
            "ENGINE OVERHEATING"
        )

elif row["fuel"] < 15:

    st.warning(
        "LOW FUEL"
    )

else:

    st.success(
        "VEHICLE HEALTHY"
    )

# =====
# LIVE METRICS
# =====

st.markdown("---")
st.header("Live Metrics")

col1, col2, col3, col4 = st.columns(4)

col1.metric(
    "Speed",
    f"{row['speed']} km/h"
)

col2.metric(
    "RPM",
    int(row["rpm"])
)

col3.metric(

```

```

        "Fuel",
        f"{row['fuel']} %"
    )

col4.metric(
    "Engine Temp",
    f"{row['engine_temp']} C"
)

# =====
# GPS
# =====

st.markdown("---")
st.header("GPS Information")

gps1, gps2 = st.columns(2)

gps1.metric(
    "Latitude",
    f"{row['latitude']:.4f}"
)

gps2.metric(
    "Longitude",
    f"{row['longitude']:.4f}"
)

# -----
# Historical Data
# -----

df = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC

```

```

LIMIT 100
"""
conn
)

df = df.sort_values("id")

df["timestamp"] = pd.to_datetime(
    df["timestamp"]
)

st.markdown("---")
st.header("Historical Telemetry")

# =====
# SPEED HISTORY
# =====

st.subheader("Speed History")

fig_speed = px.line(
    df,
    x="timestamp",
    y="speed",
    title="Speed vs Time"
)

st.plotly_chart(
    fig_speed,
    use_container_width=True
)

# =====
# RPM HISTORY
# =====

st.subheader("RPM History")

```



```
fig_rpm = px.line(
    df,
    x="timestamp",
    y="rpm",
    title="RPM vs Time"
)
```

```
st.plotly_chart(
    fig_rpm,
    use_container_width=True
)
```

```
# =====
# FUEL HISTORY
# =====
```

```
st.subheader("Fuel History")
```

```
fig_fuel = px.line(
    df,
    x="timestamp",
    y="fuel",
    title="Fuel vs Time"
)
```

```
st.plotly_chart(
    fig_fuel,
    use_container_width=True
)
```

```
# =====
# ENGINE TEMPERATURE HISTORY
# =====
```

```
st.subheader("Engine Temperature History")
```

```

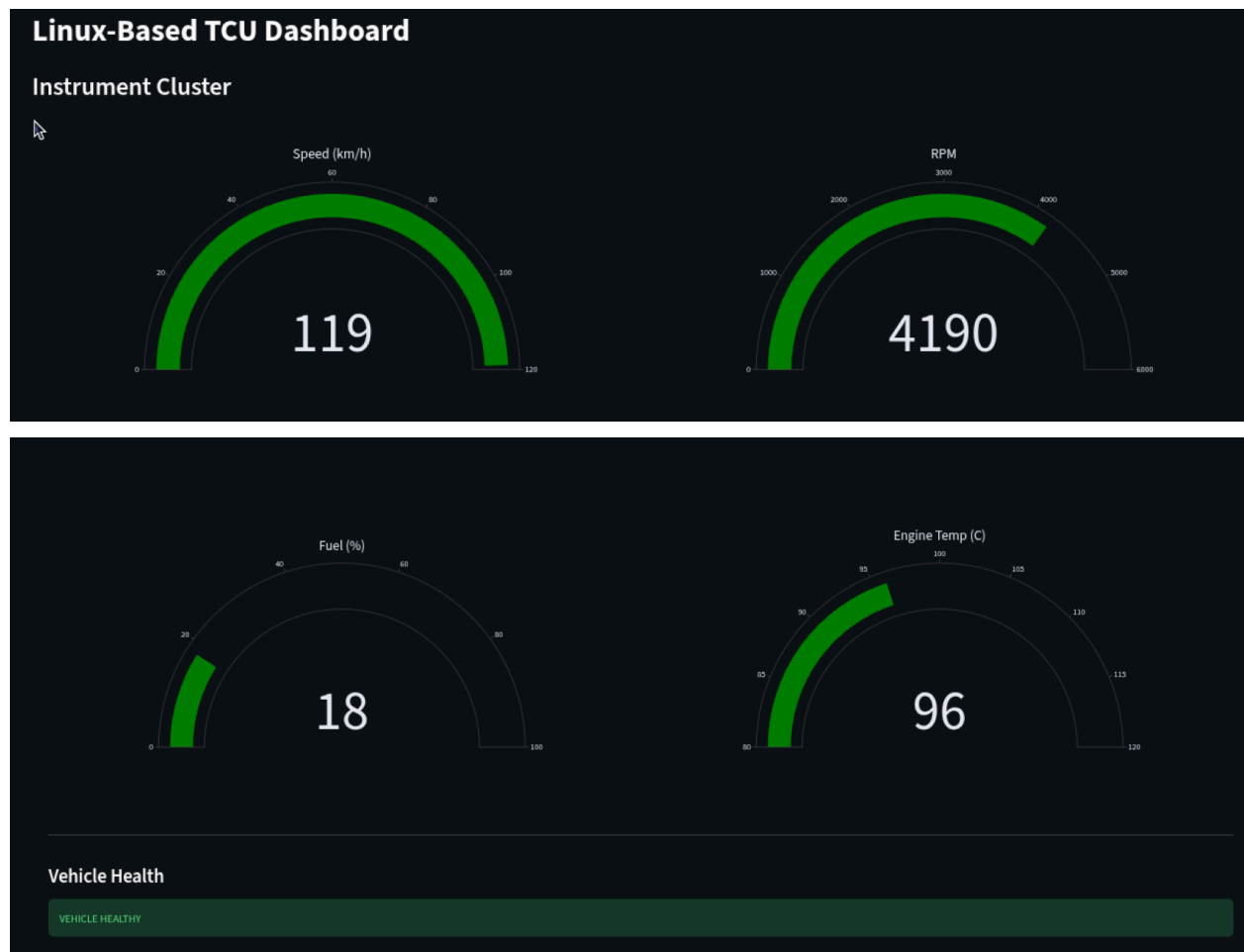
fig_temp = px.line(
    df,
    x="timestamp",
    y="engine_temp",
    title="Engine Temperature vs Time"
)

st.plotly_chart(
    fig_temp,
    use_container_width=True
)

conn.close()

```

DASHBOARD SNAPHOTS :



Live Metrics

Speed

119 km/h

RPM

4192

Fuel

18 %

Engine Temp

96 C

GPS Information

Latitude

13.0843

Longitude

80.2723

Historical Telemetry

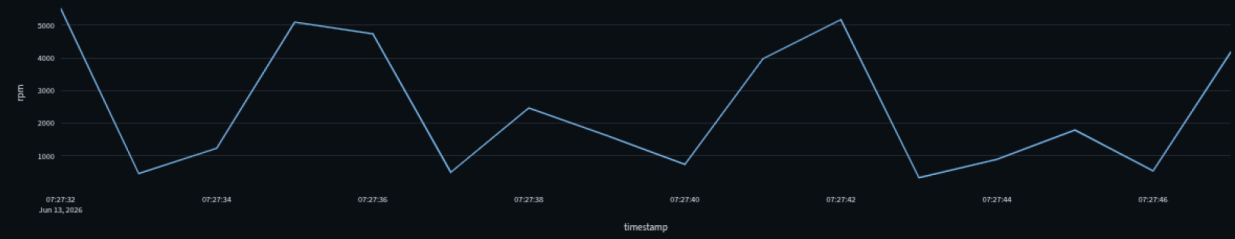
Speed History

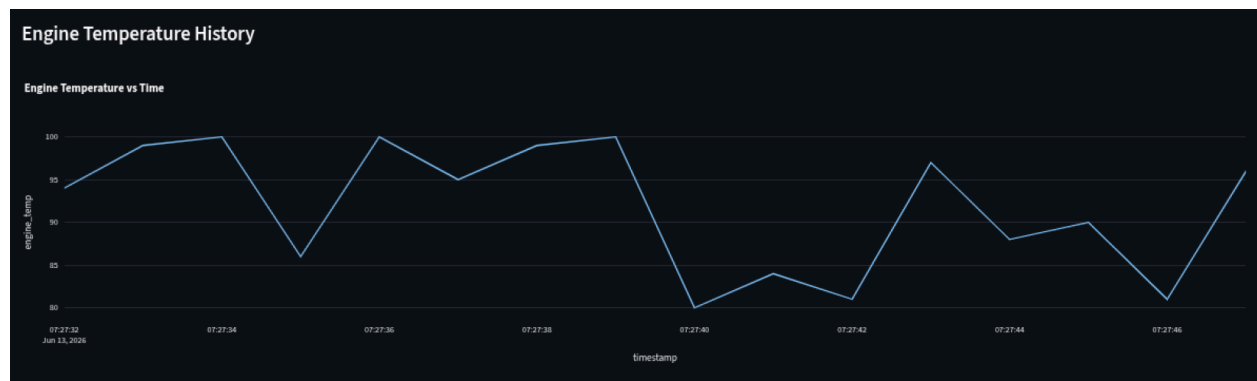
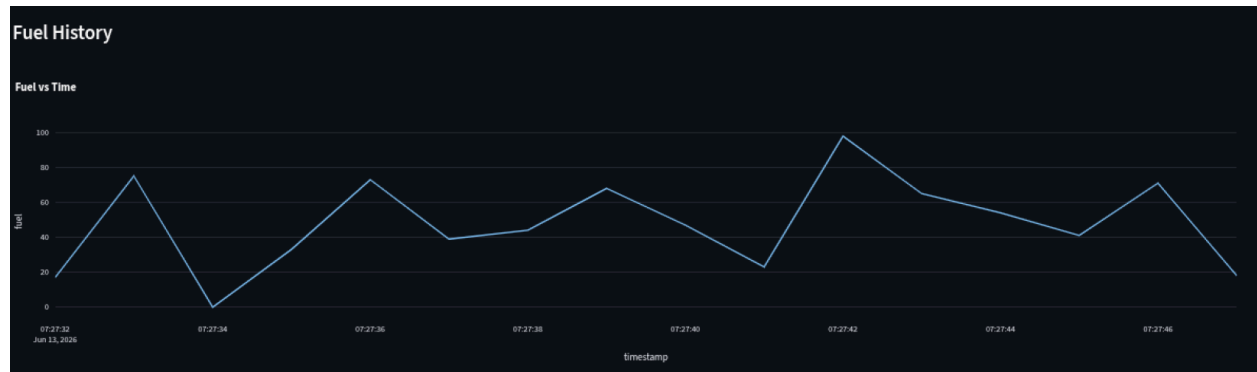
Speed vs Time



RPM History

RPM vs Time





=====

Adding Instrument Cluster to dashboard - Version 2

V2 Goal

Add colored zones:

Speed

None	
0-80	Green
80-100	Yellow
100-120	Red

RPM

None

0-4000 Green

4000-5000 Yellow

5000-6000 Red

Fuel

None

0-15 Red

15-30 Yellow

30-100 Green

Engine Temp

None

80-95 Green

95-100 Yellow

100-120 Red

AND UPGRADE :

1. Speed Gauge
2. RPM Gauge
3. Fuel Gauge
4. Engine Temperature
5. Upgrade Vehicle Health Logic

Below is the **complete updated V2 code** with all the changes integrated:

-  Green/Yellow/Red gauge zones
-  Improved vehicle health logic
-  Auto refresh
-  GPS rounding
-  Historical charts with timestamps
-  Instrument Cluster V2

TCU_dashboard_v2.py

```
import streamlit as st
import sqlite3
import pandas as pd

import plotly.express as px
import plotly.graph_objects as go

from streamlit_autorefresh import st_autorefresh

# -----
# Page Config
# -----

st_autorefresh(
    interval=1000,
    key="dashboard_refresh"
)

st.set_page_config(
    page_title="TCU Dashboard",
    layout="wide"
)

st.title("Linux-Based TCU Dashboard")

# -----
# Database Connection
# -----

conn = sqlite3.connect(
    "vehicle_telematics_v2.db"
)

# -----
# Latest Data
# -----

latest = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC
    """
)
```

```

LIMIT 1
"""
conn
)

if not latest.empty:

    row = latest.iloc[0]

    # =====
    # INSTRUMENT CLUSTER
    # =====

    st.header("Instrument Cluster")

    speed_col, rpm_col = st.columns(2)

    # -----
    # SPEED GAUGE
    # -----

    with speed_col:

        fig_speed_gauge = go.Figure(
            go.Indicator(
                mode="gauge+number",
                value=row["speed"],
                title={"text": "Speed (km/h)"},
                gauge={
                    "axis": {"range": [0, 120]},

                    "bar": {"color": "white"},

                    "steps": [
                        {"range": [0, 80], "color": "green"},
                        {"range": [80, 100], "color": "yellow"},
                        {"range": [100, 120], "color": "red"}
                    ]
                }
            )
        )

    st.plotly_chart(
        fig_speed_gauge,

```

```

        use_container_width=True
    )

# -----
# RPM GAUGE
# -----

with rpm_col:

    fig_rpm_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["rpm"],
            title={"text": "RPM"},
            gauge={
                "axis": {"range": [0, 6000]},

                "bar": {"color": "white"},

                "steps": [
                    {"range": [0, 4000], "color": "green"},
                    {"range": [4000, 5000], "color": "yellow"},
                    {"range": [5000, 6000], "color": "red"}
                ]
            }
        )
    )

    st.plotly_chart(
        fig_rpm_gauge,
        use_container_width=True
    )

# -----
# FUEL + TEMP GAUGES
# -----

fuel_col, temp_col = st.columns(2)

with fuel_col:

    fig_fuel_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",

```



```

        value=row["fuel"],
        title={"text": "Fuel (%)"},
        gauge={
            "axis": {"range": [0, 100]},

            "bar": {"color": "white"},

            "steps": [
                {"range": [0, 15], "color": "red"},
                {"range": [15, 30], "color": "yellow"},
                {"range": [30, 100], "color": "green"}
            ]
        }
    )
)

st.plotly_chart(
    fig_fuel_gauge,
    use_container_width=True
)

```

with temp_col:

```

fig_temp_gauge = go.Figure(
    go.Indicator(
        mode="gauge+number",
        value=row["engine_temp"],
        title={"text": "Engine Temp (C)"},
        gauge={
            "axis": {"range": [80, 120]},

            "bar": {"color": "white"},

            "steps": [
                {"range": [80, 95], "color": "green"},
                {"range": [95, 100], "color": "yellow"},
                {"range": [100, 120], "color": "red"}
            ]
        }
    )
)

st.plotly_chart(
    fig_temp_gauge,

```

```

        use_container_width=True
    )

# =====
# VEHICLE ALERT PANEL (V2.2 + V2.2.1)
# =====

st.markdown("---")
st.subheader("Vehicle Alerts")

alerts = []

# -----
# Engine Temperature
# -----

if row["engine_temp"] >= 100:

    alerts.append(
        "ENGINE OVERHEATING"
    )

# -----
# Fuel
# -----

if row["fuel"] <= 15:

    alerts.append(
        "LOW FUEL"
    )

# -----
# RPM
# -----

if row["rpm"] >= 5000:

    alerts.append(
        "HIGH RPM"
    )

# -----
# Speed

```

```

# -----

if row["speed"] >= 100:

    alerts.append(
        "OVERSPEED"
    )

# -----
# Display Alert Panel
# -----

if len(alerts) == 0:

    st.success(
        "VEHICLE HEALTHY"
    )

else:

    alert_text = "<br><br>".join(alerts)

    st.markdown(
        f"""
        <div style="
            border:2px solid red;
            padding:20px;
            border-radius:10px;
            background-color:#330000;
            font-size:22px;
            font-weight:bold;
        ">
        {alert_text}
        </div>
        """,
        unsafe_allow_html=True
    )

# =====
# LIVE METRICS
# =====

st.markdown("---")
st.header("Live Metrics")

```

```

col1, col2, col3, col4 = st.columns(4)

col1.metric(
    "Speed",
    f'{row["speed"]} km/h'
)

col2.metric(
    "RPM",
    int(row["rpm"])
)

col3.metric(
    "Fuel",
    f'{row["fuel"]} %'
)

col4.metric(
    "Engine Temp",
    f'{row["engine_temp"]} C'
)

# =====
# GPS INFORMATION
# =====

st.markdown("---")
st.header("GPS Information")

gps1, gps2 = st.columns(2)

gps1.metric(
    "Latitude",
    f'{row["latitude"]:.4f}'
)

gps2.metric(
    "Longitude",
    f'{row["longitude"]:.4f}'
)

# -----
# Historical Data
# -----

```

```

df = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC
    LIMIT 100
    """,
    conn
)

df = df.sort_values("id")

df["timestamp"] = pd.to_datetime(
    df["timestamp"]
)

st.markdown("---")
st.header("Historical Telemetry")

# =====
# SPEED HISTORY
# =====

st.subheader("Speed History")

fig_speed = px.line(
    df,
    x="timestamp",
    y="speed",
    title="Speed vs Time"
)

st.plotly_chart(
    fig_speed,
    use_container_width=True
)

# =====
# RPM HISTORY
# =====

st.subheader("RPM History")

```

```

fig_rpm = px.line(
    df,
    x="timestamp",
    y="rpm",
    title="RPM vs Time"
)

st.plotly_chart(
    fig_rpm,
    use_container_width=True
)

# =====
# FUEL HISTORY
# =====

st.subheader("Fuel History")

fig_fuel = px.line(
    df,
    x="timestamp",
    y="fuel",
    title="Fuel vs Time"
)

st.plotly_chart(
    fig_fuel,
    use_container_width=True
)

# =====
# ENGINE TEMPERATURE HISTORY
# =====

st.subheader("Engine Temperature History")

fig_temp = px.line(
    df,
    x="timestamp",
    y="engine_temp",
    title="Engine Temperature vs Time"
)

st.plotly_chart(

```

```
fig_temp,
use_container_width=True
)

conn.close()
```

Dashboard output :



The exact thresholds and the priority order used by the Vehicle Health Indicator.

1. Engine Temperature (Highest Priority)

Condition:

```
None  
if row["engine_temp"] >= 100:
```

Warning displayed:

```
None  
CRITICAL: ENGINE OVERHEATING
```

Gauge Zones:

```
None  
80 - 95      Green  
95 - 100     Yellow  
100 - 120    Red
```

Examples:

```
None  
94  → Green  
97  → Yellow  
100 → Red + Warning  
105 → Red + Warning
```

2. Fuel Level

Condition:

```
None  
elif row["fuel"] <= 15:
```


Warning displayed:

None

WARNING: LOW FUEL

Gauge Zones:

None

0 - 15 Red

15 - 30 Yellow

30 - 100 Green

Examples:

None

50 → Green

25 → Yellow

15 → Red + Warning

8 → Red + Warning

3. RPM

Condition:

None

```
elif row["rpm"] >= 5000:
```

Warning displayed:

None

WARNING: HIGH RPM

Gauge Zones:

None

0 - 4000	Green
4000 - 5000	Yellow
5000 - 6000	Red

Examples:

None

3500	→ Green
4500	→ Yellow
5000	→ Red + Warning
5800	→ Red + Warning

4. Vehicle Speed

Condition:

None

```
elif row["speed"] >= 100:
```

Warning displayed:

None

WARNING: OVERSPEED

Gauge Zones:

None

0 - 80	Green
80 - 100	Yellow
100 - 120	Red

Examples:

None

60 → Green

90 → Yellow

100 → Red + Warning

115 → Red + Warning

5. Healthy State

Condition:

None

else:

Displayed:

None

VEHICLE HEALTHY

Requirements:

None

Temp < 100

Fuel > 15

RPM < 5000

Speed < 100

Example:

None

Speed = 75

RPM = 3200

Fuel = 45

Temp = 92

Output:

None

VEHICLE HEALTHY

Adding GPS Tracking Map to dashboard - Version 3

V3 is probably the most visually impressive feature in your entire TCU project.

Right now you already store:

None

latitude

longitude

inside:

None

vehicle telematics_v2.db

and every second the vehicle moves:

None

vehicle.gps.latitude += 0.0001;

vehicle.gps.longitude += 0.0001;

So we already have everything needed.

V3 Goal

Add:

None

GPS Tracking Map

showing:

None

Current Vehicle Position

+

Historical Route

TCU_dashboard_v3.py

```
import streamlit as st
```

```
import sqlite3
```

```
import pandas as pd
```

```
import plotly.express as px
```

```
import plotly.graph_objects as go
```

```
from streamlit_autorefresh import st_autorefresh
```

```
# -----
```

```
# Page Config
```

```
# -----
```

```
st_autorefresh(  
    interval=1000,  
    key="dashboard_refresh"  
)
```

```
st.set_page_config(  
    page_title="TCU Dashboard",  
    layout="wide"  
)
```

```
st.title("Linux-Based TCU Dashboard")
```

```
# -----
```

```
# Database Connection
```

```
# -----
```

```

conn = sqlite3.connect(
    "vehicle_telematics_v2.db"
)

# -----
# Latest Data
# -----

latest = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC
    LIMIT 1
    """,
    conn
)

if not latest.empty:

    row = latest.iloc[0]

    # =====
    # INSTRUMENT CLUSTER
    # =====

    st.header("Instrument Cluster")

    speed_col, rpm_col = st.columns(2)

    # -----
    # SPEED GAUGE
    # -----

    with speed_col:

        fig_speed_gauge = go.Figure(
            go.Indicator(
                mode="gauge+number",
                value=row["speed"],
                title={"text": "Speed (km/h)"},
                gauge={
                    "axis": {"range": [0, 120]},

```

```

        "bar": {"color": "white"},

        "steps": [
            {"range": [0, 80], "color": "green"},
            {"range": [80, 100], "color": "yellow"},
            {"range": [100, 120], "color": "red"}
        ]
    }
)
)

st.plotly_chart(
    fig_speed_gauge,
    use_container_width=True
)

# -----
# RPM GAUGE
# -----

with rpm_col:

    fig_rpm_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["rpm"],
            title={"text": "RPM"},
            gauge={
                "axis": {"range": [0, 6000]},

                "bar": {"color": "white"},

                "steps": [
                    {"range": [0, 4000], "color": "green"},
                    {"range": [4000, 5000], "color": "yellow"},
                    {"range": [5000, 6000], "color": "red"}
                ]
            }
        )
    )

st.plotly_chart(
    fig_rpm_gauge,
    use_container_width=True
)

```

```

    )

# -----
# FUEL + TEMP GAUGES
# -----

fuel_col, temp_col = st.columns(2)

with fuel_col:

    fig_fuel_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["fuel"],
            title={"text": "Fuel (%)"},
            gauge={
                "axis": {"range": [0, 100]},

                "bar": {"color": "white"},

                "steps": [
                    {"range": [0, 15], "color": "red"},
                    {"range": [15, 30], "color": "yellow"},
                    {"range": [30, 100], "color": "green"}
                ]
            }
        )
    )

    st.plotly_chart(
        fig_fuel_gauge,
        use_container_width=True
    )

```

```

with temp_col:

    fig_temp_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["engine_temp"],
            title={"text": "Engine Temp (C)"},
            gauge={
                "axis": {"range": [80, 120]},

```



```

        "bar": {"color": "white"},

        "steps": [
            {"range": [80, 95], "color": "green"},
            {"range": [95, 100], "color": "yellow"},
            {"range": [100, 120], "color": "red"}
        ]
    }
)
)

st.plotly_chart(
    fig_temp_gauge,
    use_container_width=True
)

# =====
# VEHICLE ALERT PANEL (V2.2 + V2.2.1)
# =====

st.markdown("---")
st.subheader("Vehicle Alerts")

alerts = []

# -----
# Engine Temperature
# -----

if row["engine_temp"] >= 100:

    alerts.append(
        "ENGINE OVERHEATING"
    )

# -----
# Fuel
# -----

if row["fuel"] <= 15:

    alerts.append(
        "LOW FUEL"
    )

```

```

# -----
# RPM
# -----

if row["rpm"] >= 5000:

    alerts.append(
        "HIGH RPM"
    )

# -----
# Speed
# -----

if row["speed"] >= 100:

    alerts.append(
        "OVERSPEED"
    )

# -----
# Display Alert Panel
# -----

if len(alerts) == 0:

    st.success(
        "VEHICLE HEALTHY"
    )

else:

    alert_text = "<br><br>".join(alerts)

    st.markdown(
        f"""
        <div style="
            border:2px solid red;
            padding:20px;
            border-radius:10px;
            background-color:#330000;
            font-size:22px;
            font-weight:bold;

```

```

        ">
        {alert_text}
        </div>
        """,
        unsafe_allow_html=True
    )
# =====
# LIVE METRICS
# =====

st.markdown("---")
st.header("Live Metrics")

col1, col2, col3, col4 = st.columns(4)

col1.metric(
    "Speed",
    f"{row['speed']} km/h"
)

col2.metric(
    "RPM",
    int(row["rpm"])
)

col3.metric(
    "Fuel",
    f"{row['fuel']} %"
)

col4.metric(
    "Engine Temp",
    f"{row['engine_temp']} C"
)

# =====
# GPS INFORMATION
# =====

st.markdown("---")
st.header("GPS Information")

gps1, gps2 = st.columns(2)

```

```

gps1.metric(
    "Latitude",
    f'{row["latitude"]:.4f}'
)

gps2.metric(
    "Longitude",
    f'{row["longitude"]:.4f}'
)

# =====
# GPS MAP
# =====

st.markdown("---")
st.header("Live Vehicle Tracking")
gps_df = pd.read_sql_query(
    """
SELECT latitude, longitude
FROM telematics
ORDER BY id DESC
LIMIT 200
""",
    conn
)

gps_df = gps_df.iloc[::-1]
# -----
# Route Line
# -----

fig_map = px.line_map(
    gps_df,
    lat="latitude",
    lon="longitude",
    zoom=14,
    height=600
)

# -----
# Current Vehicle Position
# -----

current_lat = gps_df.iloc[-1]["latitude"]

```

```
current_lon = gps_df.iloc[-1]["longitude"]
```

```
fig_map.add_scattermap(  
    lat=[current_lat],  
    lon=[current_lon],  
    mode="markers",  
    marker=dict(  
        size=18  
    ),  
    name="Current Position"  
)
```

```
fig_map.update_layout(  
    margin=dict(  
        l=0,  
        r=0,  
        t=0,  
        b=0  
    )  
)
```

```
st.plotly_chart(  
    fig_map,  
    use_container_width=True  
)
```

```
# -----  
# Historical Data  
# -----
```

```
df = pd.read_sql_query(  
    """  
    SELECT *  
    FROM telematics  
    ORDER BY id DESC  
    LIMIT 100  
    """,  
    conn  
)
```

```
df = df.sort_values("id")
```

```
df["timestamp"] = pd.to_datetime(  
    df["timestamp"]
```

```

        df["timestamp"]
    )

    st.markdown("---")
    st.header("Historical Telemetry")

    # =====
    # SPEED HISTORY
    # =====

    st.subheader("Speed History")

    fig_speed = px.line(
        df,
        x="timestamp",
        y="speed",
        title="Speed vs Time"
    )

    st.plotly_chart(
        fig_speed,
        use_container_width=True
    )

    # =====
    # RPM HISTORY
    # =====

    st.subheader("RPM History")

    fig_rpm = px.line(
        df,
        x="timestamp",
        y="rpm",
        title="RPM vs Time"
    )

    st.plotly_chart(
        fig_rpm,
        use_container_width=True
    )

    # =====
    # FUEL HISTORY

```

```

# =====

st.subheader("Fuel History")

fig_fuel = px.line(
    df,
    x="timestamp",
    y="fuel",
    title="Fuel vs Time"
)

st.plotly_chart(
    fig_fuel,
    use_container_width=True
)

# =====
# ENGINE TEMPERATURE HISTORY
# =====

st.subheader("Engine Temperature History")

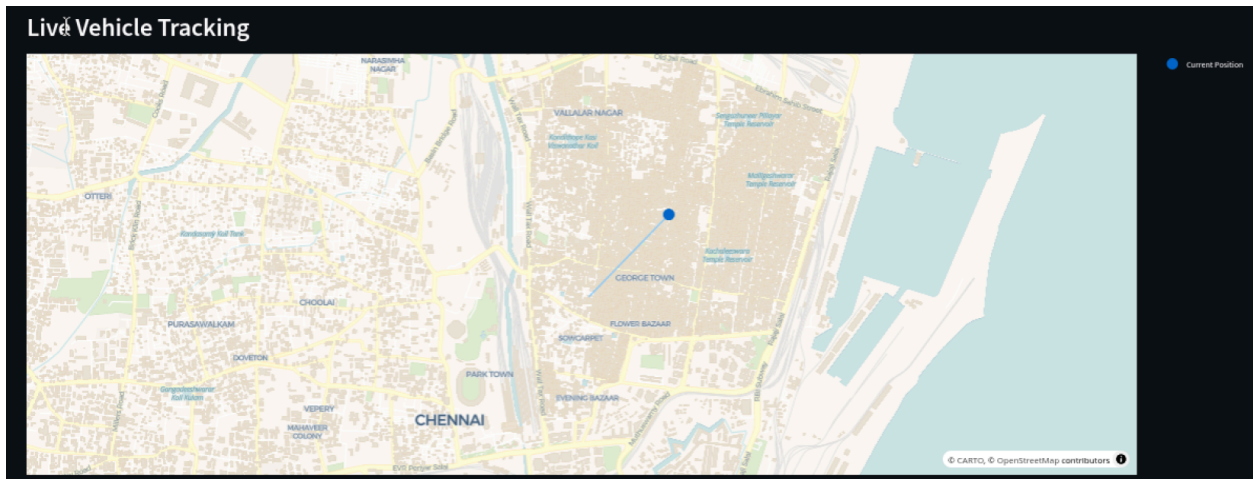
fig_temp = px.line(
    df,
    x="timestamp",
    y="engine_temp",
    title="Engine Temperature vs Time"
)

st.plotly_chart(
    fig_temp,
    use_container_width=True
)

conn.close()

```

DASHBOARD O/P:



Current Project Status

You've now completed:

None

Phase 1	Vehicle Simulator	✓
Phase 2	SocketCAN	✓
Phase 3	MQTT	✓
Phase 4	JSON Telemetry	✓
Phase 5	SQLite Logging	✓
Phase 6	Streamlit Dashboard	✓
Phase 7	Instrument Cluster V2.2	✓
Phase 8	GPS Route Tracking V3.1	✓

Architecture:

None

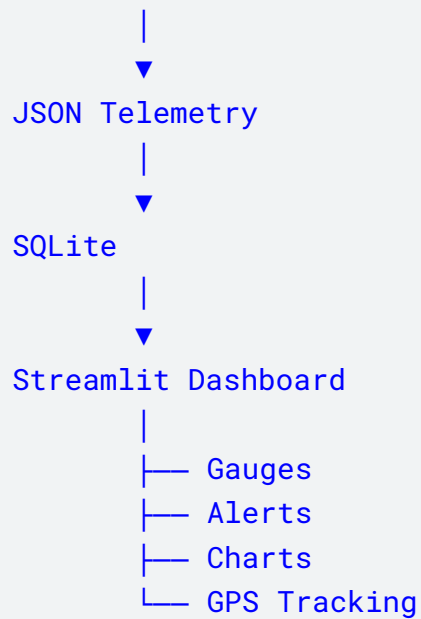
C++ Vehicle Simulator



SocketCAN



MQTT



Adding analytics to dashboard - Version 3

Current Project Status

I have implemented:

None

Phase 1	Vehicle Simulator	✓
Phase 2	SocketCAN	✓
Phase 3	MQTT	✓
Phase 4	JSON Telemetry	✓
Phase 5	SQLite Logging	✓
Phase 6	Streamlit Dashboard	✓
Phase 7	Instrument Cluster	✓
Phase 8	GPS Route Tracking	✓



Your dashboard now contains:

Instrument Cluster

- ✓ Speed Gauge
 - ✓ RPM Gauge
 - ✓ Fuel Gauge
 - ✓ Engine Temperature Gauge
-

Vehicle Monitoring

- ✓ Alert Panel
 - ✓ Live Metrics
 - ✓ GPS Coordinates
-

GPS Analytics

- ✓ Live Vehicle Tracking
 - ✓ Route Visualization
 - ✓ Distance Travelled
-

Database Analytics

- ✓ Total Records
- ✓ Average Speed
- ✓ Maximum RPM

- ✓ Average Fuel
 - ✓ Maximum Temperature
-

Historical Analysis

- ✓ Speed History
 - ✓ RPM History
 - ✓ Fuel History
 - ✓ Temperature History
-

Resume Impact

You can now honestly describe the project as:

Developed a Linux-Based Automotive Telematics Control Unit (TCU) Simulator on Raspberry Pi using C++, SocketCAN, MQTT, SQLite, and Streamlit. Implemented real-time vehicle telemetry, GPS route tracking, instrument cluster visualization, analytics dashboard, alert management, and distance-travelled estimation.

For Distance Travelled Analytics Add: **geopy lib**

Step 1: Install Geopy

On your Raspberry Pi:

None

```
pip3 install geopy
```

Verify:

None

```
python3 -c "from geopy.distance import geodesic; print('OK')"
```

Step 2: Add Import

Near the top of your dashboard:

None

```
from geopy.distance import geodesic
```

TCU_dashboard.py:

```
import streamlit as st
import sqlite3
import pandas as pd
```

```
import plotly.express as px
import plotly.graph_objects as go
```

```
from geopy.distance import geodesic
```

```
from streamlit_autorefresh import st_autorefresh
```

```
# -----
# Page Config
# -----
```

```
st_autorefresh(
    interval=4000,
    key="dashboard_refresh"
)
```

```
st.set_page_config(
    page_title="TCU Dashboard",
    layout="wide"
)
```

```
st.title("Linux-Based TCU Dashboard")
```

```
# -----
# Database Connection
# -----
```

```
conn = sqlite3.connect(
    "vehicle_telematics_v2.db"
)
```

```
#-----analytics window func-----#
```

```
@st.cache_data(ttl=10)
def get_analytics():
```

```
    return pd.read_sql_query(
        """
        SELECT
            COUNT(*) AS total_records,
            AVG(speed) AS avg_speed,
            MAX(rpm) AS max_rpm,
            AVG(fuel) AS avg_fuel,
            MAX(engine_temp) AS max_temp
        FROM telematics
        """,
        conn
    )
```

```
#-----distance calc func-----#
```

```

@st.cache_data(ttl=10)
def get_total_distance():

    gps_data = pd.read_sql_query(
        """
        SELECT latitude, longitude
        FROM telematics
        ORDER BY id ASC
        """,
        conn
    )

    total_distance = 0.0

    for i in range(1, len(gps_data)):

        point1 = (
            gps_data.iloc[i-1]["latitude"],
            gps_data.iloc[i-1]["longitude"]
        )

        point2 = (
            gps_data.iloc[i]["latitude"],
            gps_data.iloc[i]["longitude"]
        )

        total_distance += geodesic(
            point1,
            point2
        ).kilometers

    return total_distance

# -----
# Latest Data

```

```
# -----
```

```
latest = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC
    LIMIT 1
    """,
    conn
)
```

```
if not latest.empty:
```

```
    row = latest.iloc[0]
```

```
    # =====
    # INSTRUMENT CLUSTER
    # =====
```

```
    st.header("Instrument Cluster")
```

```
    speed_col, rpm_col = st.columns(2)
```

```
    # -----
    # SPEED GAUGE
    # -----
```

```
    with speed_col:
```

```
        fig_speed_gauge = go.Figure(
            go.Indicator(
                mode="gauge+number",
                value=row["speed"],
                title={"text": "Speed (km/h)"},
                gauge={
                    "axis": {"range": [0, 120]},
```

```

        "bar": {"color": "white"},

        "steps": [
            {"range": [0, 80], "color": "green"},
            {"range": [80, 100], "color": "yellow"},
            {"range": [100, 120], "color": "red"}
        ]
    }
)
)

st.plotly_chart(
    fig_speed_gauge,
    use_container_width=True
)

# -----
# RPM GAUGE
# -----

with rpm_col:

    fig_rpm_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["rpm"],
            title={"text": "RPM"},
            gauge={
                "axis": {"range": [0, 6000]},

                "bar": {"color": "white"},

                "steps": [
                    {"range": [0, 4000], "color": "green"},
                    {"range": [4000, 5000], "color": "yellow"},
                    {"range": [5000, 6000], "color": "red"}
                ]
            }
        )
    )

```



```

        ]
    }
)
)

st.plotly_chart(
    fig_rpm_gauge,
    use_container_width=True
)

# -----
# FUEL + TEMP GAUGES
# -----

fuel_col, temp_col = st.columns(2)

with fuel_col:

    fig_fuel_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["fuel"],
            title={"text": "Fuel (%)"},
            gauge={
                "axis": {"range": [0, 100]},

                "bar": {"color": "white"},

                "steps": [
                    {"range": [0, 15], "color": "red"},
                    {"range": [15, 30], "color": "yellow"},
                    {"range": [30, 100], "color": "green"}
                ]
            }
        )
    )

```

```

st.plotly_chart(
    fig_fuel_gauge,
    use_container_width=True
)

with temp_col:

    fig_temp_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=row["engine_temp"],
            title={"text": "Engine Temp (C)"},
            gauge={
                "axis": {"range": [80, 120]},

                "bar": {"color": "white"},

                "steps": [
                    {"range": [80, 95], "color": "green"},
                    {"range": [95, 100], "color": "yellow"},
                    {"range": [100, 120], "color": "red"}
                ]
            }
        )
    )

st.plotly_chart(
    fig_temp_gauge,
    use_container_width=True
)

# =====
# VEHICLE ALERT PANEL (V2.2 + V2.2.1)
# =====

st.markdown("---")
st.subheader("Vehicle Alerts")

```

```
alerts = []
```

```
# -----
```

```
# Engine Temperature
```

```
# -----
```

```
if row["engine_temp"] >= 100:
```

```
    alerts.append(
        "ENGINE OVERHEATING"
    )
```

```
# -----
```

```
# Fuel
```

```
# -----
```

```
if row["fuel"] <= 15:
```

```
    alerts.append(
        "LOW FUEL"
    )
```

```
# -----
```

```
# RPM
```

```
# -----
```

```
if row["rpm"] >= 5000:
```

```
    alerts.append(
        "HIGH RPM"
    )
```

```
# -----
```

```
# Speed
```

```
# -----
```

```

if row["speed"] >= 100:

    alerts.append(
        "OVERSPEED"
    )

# -----
# Display Alert Panel
# -----

if len(alerts) == 0:

    st.success(
        "VEHICLE HEALTHY"
    )

else:

    alert_text = "<br><br>".join(alerts)

    st.markdown(
        f"""
        <div style="
            border:2px solid red;
            padding:20px;
            border-radius:10px;
            background-color:#330000;
            font-size:22px;
            font-weight:bold;
        ">
        {alert_text}
        </div>
        """,
        unsafe_allow_html=True
    )
# =====
# LIVE METRICS

```

```
# =====
```

```
st.markdown("---")  
st.header("Live Metrics")
```

```
col1, col2, col3, col4 = st.columns(4)
```

```
col1.metric(  
    "Speed",  
    f'{row["speed"]} km/h'  
)
```

```
col2.metric(  
    "RPM",  
    int(row["rpm"])  
)
```

```
col3.metric(  
    "Fuel",  
    f'{row["fuel"]} %'  
)
```

```
col4.metric(  
    "Engine Temp",  
    f'{row["engine_temp"]} C'  
)
```

```
# =====
```

```
# GPS INFORMATION
```

```
# =====
```

```
st.markdown("---")  
st.header("GPS Information")
```

```
gps1, gps2 = st.columns(2)
```

```
gps1.metric(  
    "GPS Lat",  
    f'{row["lat"]}°N'  
)
```

```

        "Latitude",
        f'{row["latitude"]:.4f}'
    )

gps2.metric(
    "Longitude",
    f'{row["longitude"]:.4f}'
)

# =====
# GPS MAP
# =====

st.markdown("---")
st.header("Live Vehicle Tracking")
gps_df = pd.read_sql_query(
    """
SELECT latitude, longitude
FROM telematics
ORDER BY id DESC
LIMIT 50
""",
    conn
)

gps_df = gps_df.iloc[::-1]
# -----
# Route Line
# -----

fig_map = px.line_map(
    gps_df,
    lat="latitude",
    lon="longitude",
    zoom=14,
    height=600
)

```

```
# -----  
# Current Vehicle Position  
# -----
```

```
current_lat = gps_df.iloc[-1]["latitude"]  
current_lon = gps_df.iloc[-1]["longitude"]
```

```
fig_map.add_scattermap(  
    lat=[current_lat],  
    lon=[current_lon],  
    mode="markers",  
    marker=dict(  
        size=18  
    ),  
    name="Current Position"  
)
```

```
fig_map.update_layout(  
    margin=dict(  
        l=0,  
        r=0,  
        t=0,  
        b=0  
    )  
)
```

```
st.plotly_chart(  
    fig_map,  
    use_container_width=True  
)
```

```
# =====
```

```
# ANALYTICS SUMMARY
```

```
# =====
```

```
st.markdown("---")
```

```
st.header("Analytics Summary")
```

```
analytics = get_analytics()
```

```
distance_km = get_total_distance()
```

```
stats = analytics.iloc[0]
```

```
a1, a2, a3, a4, a5, a6 = st.columns(6)
```

```
a1.metric(
    "Total Records",
    int(stats["total_records"])
)
```

```
a2.metric(
    "Average Speed",
    f"{stats['avg_speed']:.1f} km/h"
)
```

```
a3.metric(
    "Maximum RPM",
    int(stats["max_rpm"])
)
```

```
a4.metric(
    "Average Fuel",
    f"{stats['avg_fuel']:.1f} %"
)
```

```
a5.metric(
    "Maximum Temp",
    f"{stats['max_temp']} C"
)
```



```
a6.metric(
    "Distance Travelled",
    f"{distance_km:.2f} km"
)
```

```
# -----
# Historical Data
# -----
```

```
df = pd.read_sql_query(
    """
    SELECT *
    FROM telematics
    ORDER BY id DESC
    LIMIT 50
    """,
    conn
)
```

```
df = df.sort_values("id")
```

```
df["timestamp"] = pd.to_datetime(
    df["timestamp"]
)
```

```
st.markdown("---")
st.header("Historical Telemetry")
```

```
# =====
# SPEED HISTORY
# =====
```

```
st.subheader("Speed History")
```

```
fig_speed = px.line(
```

```
df,  
x="timestamp",  
y="speed",  
title="Speed vs Time"  
)
```

```
st.plotly_chart(  
    fig_speed,  
    use_container_width=True  
)
```

```
# =====  
# RPM HISTORY  
# =====
```

```
st.subheader("RPM History")
```

```
fig_rpm = px.line(  
    df,  
    x="timestamp",  
    y="rpm",  
    title="RPM vs Time"  
)
```

```
st.plotly_chart(  
    fig_rpm,  
    use_container_width=True  
)
```

```
# =====  
# FUEL HISTORY  
# =====
```

```
st.subheader("Fuel History")
```

```
fig_fuel = px.line(  
    df,
```

```

        x="timestamp",
        y="fuel",
        title="Fuel vs Time"
    )

    st.plotly_chart(
        fig_fuel,
        use_container_width=True
    )

# =====
# ENGINE TEMPERATURE HISTORY
# =====

    st.subheader("Engine Temperature History")

    fig_temp = px.line(
        df,
        x="timestamp",
        y="engine_temp",
        title="Engine Temperature vs Time"
    )

    st.plotly_chart(
        fig_temp,
        use_container_width=True
    )

    conn.close()

```

Current Analytics Summary:

None

Total Records	147
Average Speed	62.9 km/h

Maximum RPM	5983
Average Fuel	52.4 %
Maximum Temp	100.0 C
Distance Travelled	2.48 km

The distance value is actually very believable:

None

147 telemetry samples
≈ 16–17 meters per GPS update
≈ 2.3–2.5 km total distance

which aligns almost perfectly with your:

None

```
vehicle.gps.latitude += 0.0001;  
vehicle.gps.longitude += 0.0001;
```

simulation.